Computational Schematic Design Utilizing Self-Organizing Programmatic Agents

A novel approach to visualizing and organizing urban and architectural data

Nicholas A. van Son[1], Marshall Prado[2]
[1,2]University of Tennessee Knoxville, College of Architecture and Design
[1]nvanson@vols.utk.edu [2]marshall.prado@utk.edu

Architectural design requires the negotiation of a wide variety of often conflicting constraints and conditions. This puts a tremendous burden on designers to understand and evaluate all the design and site parameters in the conceptual phase of the project. Design methodologies that utilize conventional means of representation such as site diagrams, maps, or other orthographic projections may not be adequate to produce truly integrative design solutions. They often simplify conditions for user clarity or eliminate volumetric and temporal data entirely. As computational design tools develop and the mapping of georeferenced urban data becomes more commonplace, it becomes possible to integrate spatial information into design strategies and evaluate various relationships more effectively. Taking clues from medical imaging, voxel data is used to represent volumetric gradients in material properties and densities of spatial conditions. This method can be used to generate morphogenic spatial analysis of an existing site. The research presented here explores how self-organizing programmatic agents can use this analysis and embedded behaviors to visualize performative schematic design scenarios. These agents, which represent a variety of functional spaces, programmatic requirements, design constraints, and value sets, can negotiate the myriad of environmental and socio-economic site conditions as well as interact with other adaptive programmatic spaces. Each agent can iteratively search for the space that best suits the desired conditions of its program. Various agents compete for space so the overall performance of the spatial arrangement is maximized. This self-organizing spatial system presents a novel and viable means for designers to more effectively implement both urban data and computational design methods into architectural design scenarios.

Keywords: *Agent-based Modeling, Voxels, Generative Design, Self-Organizing, Urban Data Mapping, Optimization, Spatial Analysis*

INTRODUCTION

As the built environment continues to expand and site conditions become increasingly dense with environmental and socio-economic data, modern architects struggle to comprehend and negotiate the numerous parameters surrounding the conceptual phase of a design project. Without proper consideration of surrounding spatial properties, subsequent design phases may suffer from preemptive decisions and limited design potential. Conventional design methodologies and

representational techniques either hinder the architect's consideration of these properties or may require new methods and tools to fully integrate them into a data-driven design workflow.

Traditional representational methods of mapping, diagramming, or using orthographic projections have long been used to highlight important factors and conditions of a site. Foregrounding clarity and legibility in these representational artifacts often comes with a necessary yet subjective filtering of data. At the same time, an exponential increase in urban data paired with architecture's relationship to complex environmental, social, economic, or political conditions means that traditional representation and design strategies may no longer adequately account for the majority of variables that a designer needs to consider. This is not to say that mapping strategies have not matured with digital technology, but rather that they have yet to take full advantage of a modern designer's digital toolbox. Additionally, designers rely on these inadequate representations to understand the context and make design decisions that may impact the development of a project. Integrated processes that generatively combine analysis and design can lead to more performative design strategies.

New representational methods and metrics for analysis are required to synthesize these complex ideas and implement them into design strategies. Recent research has shown this to be possible by utilizing 3D voxelized representations such as those found in the medical field in Computer Tomography (CT) and Magnetic Resonance Tomography (MRI) (Patil and Ravi, 2005). These voxel methods can effectively display various categories of data in volumetric space through the use of image, color, and brightness gradients. The use of voxels within computer-aided design (CAD) helps to standardize the storage of 3D data in a way that can easily change in scale and resolution. Having a voxelized metric of space that is embedded with various forms of spatial data enables integrated spatial analysis of dense site conditions. This research works to build upon these methods to refine the designer's control to an even greater degree. The voxel space also provides a framework for generative and self-organization strategies to be implemented, drawing upon logic's found in computational systems like cellular automata and agent-based modeling where neighboring conditions can create changes in state or autonomous agents imbued with custom behaviors can negotiate their digital environment or relationships with other agents (Gardner, 1970) (Chen, 2012). These processes can be reinterpreted to control relationships between various programs and integrate analytical design tools into the schematic phase of a project. This research works to develop these tools which will allow the designer to (1) collect data according to design parameters, performative site conditions, and project requirements, (2) analyze and weigh each of the collected datatypes, and (3) simulate, negotiate, and influence the organization of program according to the analysis, programmatic relationships, and aesthetic judgement. This will improve the designer's ability to manage dense site conditions and make informed schematic design decisions.

CONTEXT

Architecture and urban design workflows have yet to fully capitalize on generative design strategies based on complex spatial data. With the standardization and proliferation of geospatial data and tools such as ArcGIS, OpenStreetMap (OSM), Google Earth, and the United States Geological Survey (USGS), geospatial data can be implemented into existing representational workflows. However, all of these still rely on 2D and 2.5D projections of geometry (Miüller, 2020). These projections often simplify, generalize or eliminate volumetric, spatial, and temporal data.

On the other hand, there are several innovative projects which begin to introduce computational approaches to schematic architectural workflows. In Reeves and Meyboom's (2013, p. 199) work, "The task of space planning, one that is typically carried out by a singular high-level decision-maker (the

architect), is approached through the distributed decision-making of low-level collective intelligence." While this research shifts the organizational decision-making process to a generative computational tool, one which utilizes the interaction of multiple agents, it does not account for other external environmental and spatial criteria. Alternatively, MVRDV's (2007) "Space Fighter: The Evolutionary City (Game:)", while not agent-based, describes a simulation tool that is built on the evolution, biology, sociology, and economics of a city. This invokes a spatial ecology where multiple interrelated spatial criteria affect urban development. More recent models, like Stanislas Chaillou's (2019) thesis *AI + Architecture: Towards a New Approach*, introduce artificial intelligence and generative adversarial networks (GAN) to floorplan analysis and generation. This method can autonomously synthesize complex solutions but relies on training data based on existing plans rather than performance criteria and user control. Although these examples work to either develop generative workflows or to collect, analyze and visualize spatial urban data, none combine the two into a single integrated design tool. Integrating intermediate stages of user input may offer more control and understanding of generative processes and complex spatial data. The need for more effective analytical and generative tools in architecture presents an opportunity to develop custom processes for schematic design workflows.

Data interpretation

Urban data is necessary for understanding site context and spatial performance, but considering that collections of this data are everchanging and growing, a standard analytical approach is not sufficient. A more effective approach to collecting and understanding urban data can take advantage of modern CAD and scripting software to help visualize site conditions in a spatial context.

Because modern site conditions involve data of varying types, sources for this data will likely vary across multiple databases and platforms. Additionally, different resources may or may not be available for different geographic locations, meaning that certain data types can not always be considered or will have to be generated from alternative methods. These variabilities force the designer across multiple platforms and visualizations, making it no less difficult to make well-informed schematic design decisions.

To effectively analyze the dense urban conditions of a site, the designer needs to be able to interpret the data by a common metric. The use of CAD and visual scripting software provides the necessary flexibility in reading various data types, formats, and sources. This allows for the data to be interpreted and visualized spatially, therefore enabling a more comprehensible representation of the site conditions. The formatting of this data and its collection into a single visualization also means that specific values can be weighted and evaluated on a per-project and site-specific basis (Prado, 2019). Through the collection of data, its interpretation, and its evaluation, the designer has multiple points of interaction where he or she can weigh, filter, and tailor the process to customize the output.

Generative design

The iterative logic presented in studies of cellular automata theory and agent-based modeling was originally developed in mathematics and has proved useful in various fields such as physics, theoretical biology, and microstructure modeling. More recently, advanced computational processes and generative design tools have enabled new self-forming architectures and AI applications in architectural practices (Autodesk, 2022). With the growing familiarity of these logics across multiple fields, it becomes increasingly easy to embed them within a design methodology. Doing so can introduce the added benefits of computational iteration and combine a customizable analytical system with the architect's awareness of programmatic needs.

Iteration is an important part of developing design proposals. This is typically a slow feedback

process of designing, evaluating, and redesigning. However, with computational tools, hundreds or thousands of iterations can be developed not only to test alternative designs in a traditional sense, but also to incrementally find more optimized solutions which analyze, visualize, and negotiate complex scenarios.

An agent-based, self-organizing architectural system was developed that could negotiate complex spatial conditions, user inputs, and behaviors for programmatic performance. This system was tested within a case study project where various site and programmatic conditions could be explored within the context of schematic design.

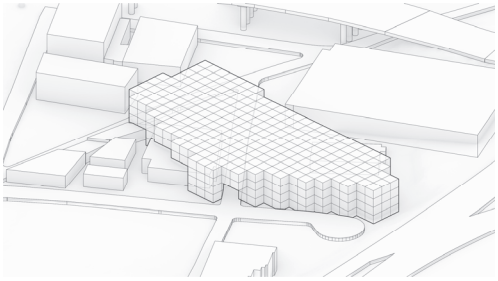

Figure 1
Maximum Buildable Volume (MBV)

METHODS

Rhinoceros 3D and Grasshopper were the primary software used throughout this investigation, serving as a standard CAD program for modeling and data visualization, and as a textural interface for custom tools and processes. ArcGIS was initially used to collect specific site data from compiled databases which was exported for use in Rhino. With Rhino serving as the primary visualization platform, Grasshopper scripts and custom Python components could perform iterative analysis and generate programmatic organizations according to the design criteria defined by the user.

Data

To test a series of self-organizing programmatic agents, an area in Knoxville, Tennessee's Warehouse district was chosen as an initial site. This area, though very close to downtown, is primarily empty lots with a few scattered buildings and large-scale highway infrastructure. Because of the proximity to the center of Knoxville and the significant GIS resources complied by the municipality, multiple types of data could be collected for the site. These data types included building locations, land use, crime distribution, local roads, highways, and railroads. In the 3D modeling environment, additional data and spatial relationships could be analyzed and created as needed. Categories of urban data that were analyzed include sun exposure, prevailing wind direction, and topographic slope. This data was imported or stored in the modeling software as geometry, point locations, and vector directions. This collection of data is representative of a variety of potentially conflicting design conditions and can be tailored to aesthetic and performance criteria specific to the user and project.

Voxels

A metric by which data could be analyzed was needed to combine all the various spatial categories. A volumetric envelope referred to as the maximum buildable volume (MBV) (see figure 1) was determined for the site. This volume represents the extent of the analysis mapping and visualization, though the surrounding context was used for calculations. Zoning restrictions for maximum building height, infrastructure easements, and setbacks were used to determine the overall form of the MBV. This volume was populated with voxels according to the resolution of the analysis. In this context, this is determined by a balance between the designer's desired resolution and hardware and software limitations. For example, if performing an analysis at the scale of the city, only a coarse voxel grid may be required. The voxel resolution may increase when analyzing a smaller site or building volume. The voxelized landscape of the MBV creates a basis on which the collected data can be interpreted, applied, and mapped.

Interpretation of the data is done in many ways depending on the datatype. For data that is

concerned with proximity, distance is calculated to the nearest point, line, or other geometry used in the calculation. Geometric criterion such as topography slope is calculated in Rhino. Volumetric data in this case can be associated with the topography below each voxel and calculated as a list of angles from the normal vector of the surface model of the site. Values for each datatype, whether distance, angle, or another unit, are remapped from -1 to 1 so that all values are relative to one another and can be compared. These unitless values can be combined or mapped to a single gradient (see figure 2). This data can be evaluated volumetrically at any location within the MBV. Each voxel receives an independent value per analyzed criteria that is contextual and comparable to other voxels in the area.

While the summation of all the criteria can generate a comprehensive voxel gradient of heterogeneous urban properties, a user valuation of individual criteria is used to weigh the data according to site and project-dependent requirements. The valuation process provides some user feedback and agency in the design process which can control autonomous agent behavior and programmatic organization.

Agents

With an MBV and the voxel gradient established, programmatic agents were developed to navigate the digital landscape, consider programmatic requirements, synthesize user preferences, and negotiate other programmatic entities simultaneously. Performance preferences for each program concerning each dataset were used to determine if specific voxels are beneficial or should be avoided. By combining the weighted value for every data type and every program, an optimal starting location can be found. This is determined by selecting the highest performing voxel from the total ranking of all the space in the MBV. Subsequently, each programmatic agent can claim more adjacent voxels as needed until it reaches the desired square footage of each program. The optimal square footage can also be unique per program. Voxels are claimed systematically considering performance and adjacency. With the implementation of various behaviors and interactions, the system can reach an organization where each program can negotiate its voxel locations within a schematic design.

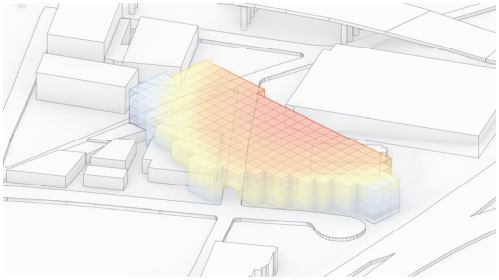

Figure 2
Values Mapped to MBV Gradient

Behavior. While the same algorithm is used to determine the behavior of every program, varying value sets and performance preferences enables each agent to grow uniquely. With the starting location determined, an agent finds its neighboring voxels and determines the one with the highest value. This highest value neighbor is added to the agent's list of claimed spaces. This is repeated for every programmatic agent and iterated indefinitely. When agents contain more than one voxel, every voxel neighbor is analyzed.

When the number of voxels within an agent reaches the program's limit, defined by its square footage, the agent will no longer increase in size. To maintain this size while continuing the iterative analysis, the highest valued neighbor is added, and the lowest valued claimed voxel is removed (see figure 3). Through this process, the overall program can traverse the MBV and change shape to reach an overall state of equilibrium and optimization.

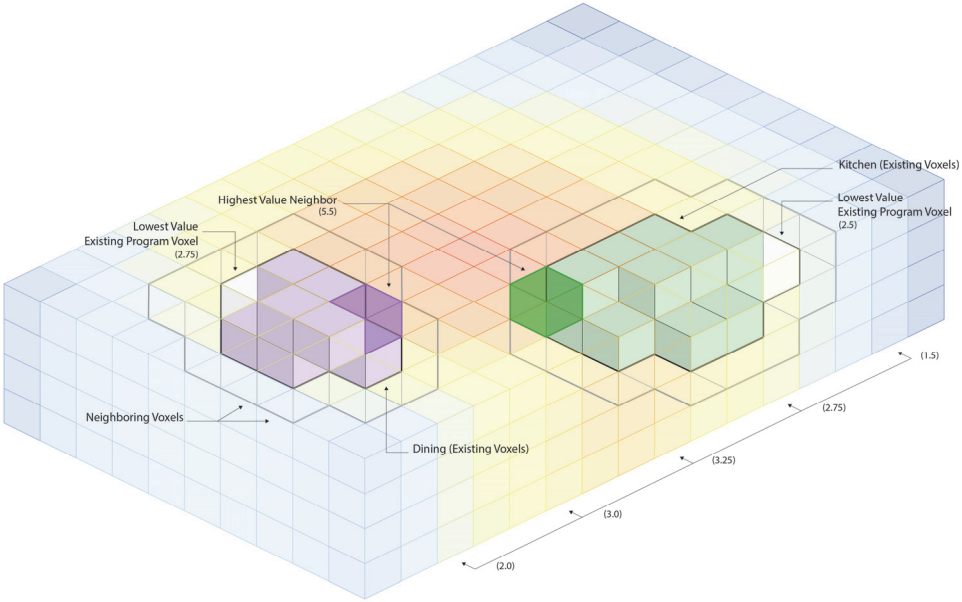

Figure 3
Program Growth Behavior (Attracted)

Interaction. In addition to the shared algorithmic behaviors across the various programs, each agent also has unique characteristics that will affect how it organizes itself within the space and around the other programs. These characteristics build directly upon the values determined in the gradient.

The designer can assign each programmatic agent a series of binary values according to which other programs he or she thinks it should avoid or seek out. For example, if it is determined that a kitchen program should be near the dining space, it would receive a positive value (True) for dining. This would control the agent's behavior to seek out the dining agent and to be near it. If it is preferred that the kitchen program should avoid being near the front entrance then it would receive a negative value (False) for the entryway program. With these relationships considered for every program, the simulation has a completely new way of resolving itself.

With each iteration, the values for every program are updated according to how the agents have grown and moved. This way, the analysis is always being performed according to the most up-to-date program organization. As iterations continue, the values for each program will eventually reach a performance equilibrium.

While the use of an agent-based system implies that there is a final optimized organization of the program, that is not to say that a designer would perform only one analysis and receive only one result. With this tool, the designer can restructure each program's relationships, running the analysis multiple times until reaching a result that is desired both aesthetically and spatially.

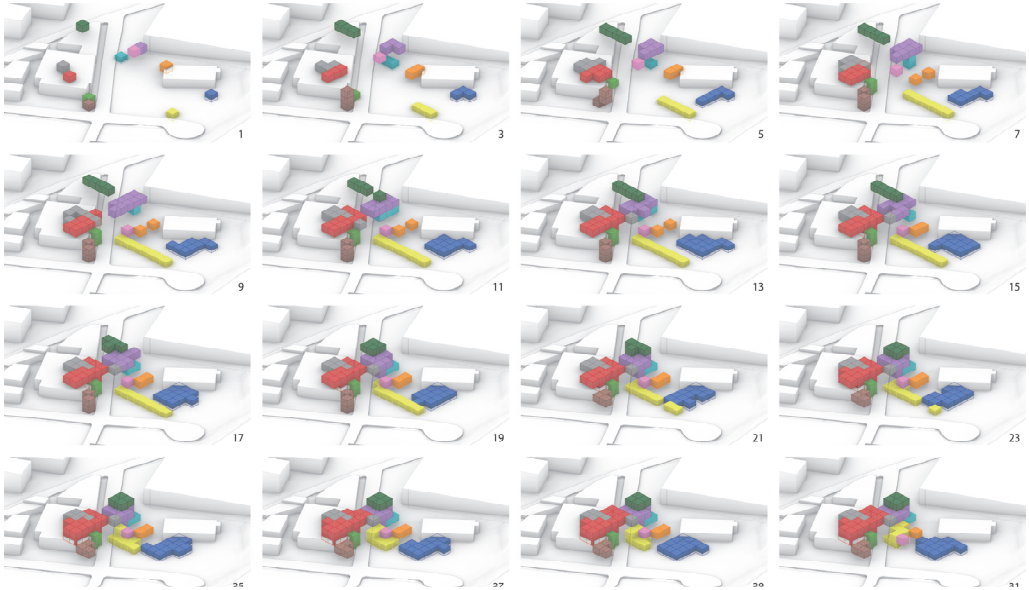

Figure 4
Real-Time Agent Organization Iterations

RESULTS

The Knoxville, Tennessee case study yielded several iterations of results that successfully negotiated space across a complex inventory of programmatic agents, all while accounting for site conditions, project-specific requirements, and designer preferences. Custom Python scripts provided the ability to watch the agents organize themselves in real-time (see figure 4). This allowed for a better understanding of how the programs interacted with one another and helped to debug the analysis from both a script development standpoint and the perspective of the designer.

Not all parameters set by the designer can achieve a result that meets both aesthetic and performative criteria. Preferred solutions could be achieved, but not always. The customizability of programmatic preferences and the weighing of site conditions allowed for numerous program organization strategies to be tested to find results that were preferred (see figure 5). An interesting consequence of the described self-forming system is that the processes often challenged the designer's preconceived notions of space and performance. Since the design intent was not formulated as an initial concept or formal representation, but rather instantiated through several stages of abstract user input and interaction, the resultant spatial arrangements and programmatic organizations were often novel discoveries.

DISCUSSION AND OUTLOOK

While the programmatic agents proved successful in following their behaviors and negotiating a wide variety of spatial conditions, the system is not exhaustive. Many other parameters can be added to the analysis or integrated into the agent behaviors to produce a more robust version of this tool. The setup, however, provides an adaptive framework for this to be added in the future. The process also requires practice to understand how early preferences impact the final solutions. This was perhaps because the analysis was broken up into what could be seen as several stages – data collection, MBV creation, gradient mapping, and agent simulation. In their current state, these stages

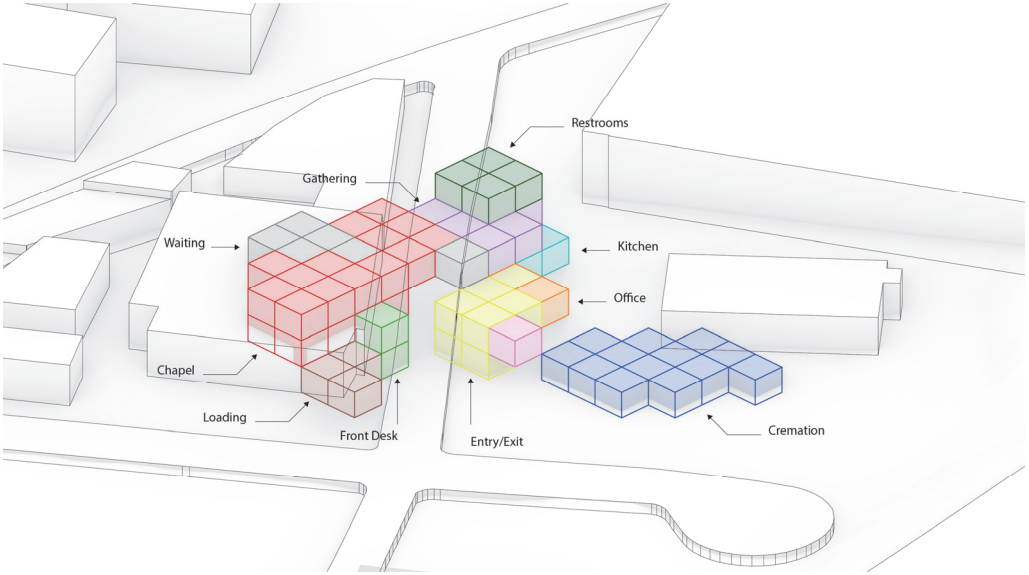

Figure 5
Final Programmatic Agent Organization

are fairly separate and sometimes involve the user returning to previous stages as a result of undesirable or under-constrained outcomes. The most frequently revisited stages are the MBV creation and gradient mapping. There are a couple of reasons that contributed to this cyclical process, most notably the starting locations for the programmatic agents. While these locations are optimal according to the values collected from the site conditions, they may not be optimal for the agent simulation. For example, by setting these locations purely from the site conditions without accounting for program preferences, certain agents may be blocked from finding the other programs it prefers to be near. Future versions of this tool would likely do better to implement a balance between site conditions and neighboring preferences at every stage of the process. Another possible solution involves refining the MBV's resolution across scales of city, site, building, and program that accommodate stalemates in the simulation. While it is important to set the bounds of the simulation, a more dense MBV provides more freedom of movement for each agent (see figure 6). Furthermore, subsets of spaces can be analyzed at an even finer voxel resolution in order to more specifically represent program areas. The case study simulations were performed at a scale which only roughly outlined spaces. Further development of higher resolution voxels could be used not only to visualize high performance spaces and locations, but also enable a greater formal understanding of the spatial data and self-forming programmatic agents.

Additionally, while the tool provides customizability within every stage of the process, some stages feature less control than others, specifically the setting of neighboring preferences. In its current state, the user only sets binary values symbolic of desiring vs avoiding. Further development of this tool would benefit from the users' ability to weigh these values, similar to the customizability of the site conditions.

Despite the need for further development and the benefits of additional user control, the

generative analysis performed here was still able to resolve an organization that implemented specific design criteria after only several tests. The current setup provides the designer with enough agency to significantly influence the outcome and allows him or her to more effectively account for the plethora of variables that exist in each site. Lastly, in addition to the designer's influence, the autonomous system plays a unique role in the design which is not experienced in many other workflows. The interplay between the designer and the agents balances the decision-making between the high-level architect and low-level agents that proved beneficial for early design development strategies.

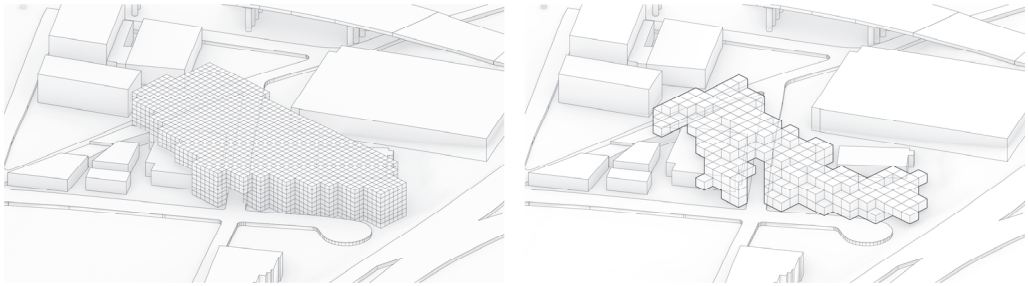

Figure 6
(Left) High Resolution MBV
(Right) Fragmented MBV

REFERENCES

Autodesk. (2022) *Generative Design for Architecture, Engineering & Construction*. Available at: https://www.autodesk.com/solutions/generative-design/architecture-engineering-construction

Chaillou, Stanislas. (2019) AI + Architecture, Thesis, Harvard GSD, Boston. Available at: http://stanislaschaillou.com/articles.html (Accessed: 28 March 2022)

Chen, L. (2012) Agent-based modeling in urban and architectural research: *A brief literature review. Frontiers of Architectural Research*, 1(2), pp.166-177.

Esri. (2020) ArcGIS 10.8.1, Redlands, USA. Available at: https://www.arcgis.com/

Gardner, Martin. (1970) "The fantastic combinations of John Conway's new solitaire

game 'life'". Mathematical Games. *Scientific American*. Vol. 223, no. 4. pp. 120-123. DOI: 10.1038/scientificamerican1070-120

Meyboom, AnnaLisa and Reeves, Dave. (2013) "Stigmergic Space." *Proceedings of the 33rd Annual Conference of the Association for Computer Aided Design in Architecture (ACADIA)*, 13, pp. 26-25.

Miüller, J.C., Weibel, R., Lagrange, J.P. and Salgé, F. (2020) Generalization: state of the art and issues. *GIS and Generalization*, pp. 3-17.

Nagy, D., Lau, D., Locke, J., Stoddart, J., Villaggi, L., Wang, R., Zhao, D. and Benjamin, D. (2017) Project Discover: An Application of Generative Design for Architectural Space Planning. *In Proceedings of the Symposium on Simulation for Architecture and Urban Design* (pp. 59-66).

Patil, S. and Ravi, B. (2005) "Voxel-based Representation, Display and Thickness Analysis of Intricate Shapes," *Ninth International Conference on Computer Aided Design and Computer Graphics* (CAD-CG'05), DOI: 10.1109/CAD-CG.2005.86

Prado, Marshall. (2019) "Morphogenic Spatial Analysis: A Novel Approach for Visualizing Volumetric Urban Conditions and Generating Analytical Morphology." *Technology | Architecture + Design*, 3:1, pp. 65-75, DOI: 10.1080/24751448.2019.1571827

Robert McNeel & Associates. (2018) Rhinoceros 6, SR14, Seattle, USA. https://www.rhino3d.com/

Souza, Eduardo. (2020) "How Will Generative Design Impact Architecture?" *ArchDaily*. [online]. Available at: https://www.archdaily.com/937772/how-will-generative-design-impact-architecture

Winy Maas, MVRDV, Delft School of Design. (2007) Space Fighter: The Evolutionary City (Game:). Barcelona: Actar

Zhang, J.H. (2021) "Urban planning and design strategy based on ArcGIS and application method." *E3S Web of Conferences*, Vol. 236, no. 03032, EDP Sciences.